Chapter 14: Indexing in Database[1]

Abu Raihan Mostofa Kamal

Professor, CSE Department
Islamic University of Technology (IUT)
Gazipur-1704 Bangladesh
raihan.kamal@iut-dhaka.edu
+8801843925543

March 3, 2026

[1] Chapter 14 (Main Textbook) and parts from other text books

Chapter Outline

Indexing: Basic Concept

Index Types: Primary and Secondary Index

Index Types: Dense and Sparse Index

B Tree

B Tree Insertion

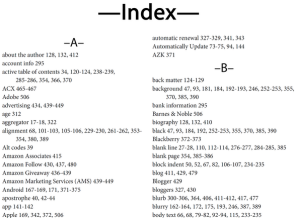
B Tree Deletion

B+ Tree

Basic Idea

Indexing

An index for a file in a database system works in much the same way as the index in this textbook. If we want to learn about a particular topic (specified by a word or a phrase) in this textbook, we can search for the topic in the index at the back of the book, find the pages where it occurs, and then read the pages to find the information for which we are looking.



—Index—

–A–

about the author 128, 132, 412
account info 295
active table of contents 34, 120-124, 238-239, 285-286, 354, 366, 370
ACX 465-467
Adobe 506
advertising 434, 439-449
age 312
aggregator 17-18, 322
alignment 68, 101-103, 105-106, 229-230, 261-262, 353-354, 380, 389
Alt codes 59
Amazon Associates 415
Amazon Fellow 430, 437, 480
Amazon Giveaway 436-439
Amazon Marketing Services (AMS) 439-449
Android 167-169, 171, 371-375
apostrophe 40, 42-44
app 141-142
Apple 169, 342, 372, 506
automatic renewal 327-329, 341, 343
Automatically Update 73-75, 94, 144
AZK 371

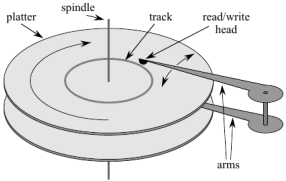
–B–

back matter 124-129
background 67, 93, 181, 184, 192-193, 246, 252-253, 355, 370, 385, 390
bank information 295
Barnes & Noble 506
biography 128, 132, 410
black 67, 93, 184, 192, 252-253, 355, 370, 385, 390
Blackberry 372-373
blank line 27-28, 110, 112-114, 276-277, 284-285, 385
blank page 356, 385-386
block indent 50, 52, 67, 82, 106-107, 234-235
blog 411, 428, 479
Blogger 429
bloggers 327, 430
blurb 300-306, 364, 406, 411-412, 417, 477
blurry 162-164, 172, 175, 193, 246, 387, 389
body text 66, 68, 79-82, 92-94, 113, 233-235

Background: Data Structure on Secondary Storage

- The primary memory (or main memory) of a computer system normally consists of silicon memory chips. It is expensive and faster.
- Most computer systems also have secondary storage based on solid-state drives (SSD) or magnetic disk drives. In recent years SSD have become cheaper with lower price.

- Primary memory is not suitable to handle database records because of its large volume.
- Because of the **moving parts (i.e. platters, arms)** in disk drives, it is slower than primary memory by a number of factors.
- Hence indexing is essential and critical for handling database records in a real-life system.



Simple Indexing will not work

Implementing an index on the student relation by keeping a sorted list of students' ID *would not work well on very large databases*.

Because:

1. The index would itself be very big
2. Even though keeping the index sorted reduces the search time, finding a student can still be rather time-consuming, and
3. Updating a sorted list as students are added or removed from the database can be very expensive

Hence, **more sophisticated indexing techniques** are used in database systems.

Indexing: Broad Classification

There are two basic kinds of indices:

1. **Ordered indices**. Based on a sorted ordering of the values.
2. **Hash indices**. Based on a uniform distribution of values across a range of buckets. The bucket to which a value is assigned is determined by a function, called a **hash function**.

Note: Concentration will be on ordered indexing.

Indexing Technique: Which one to use?

No one technique is the best. Rather, each technique is best suited to particular database applications. Each technique must be evaluated on the basis of these factors:

1. **Access types:** The types of access that are supported efficiently. Access types can include finding records with a **specified attribute value** and finding records whose attribute values fall in **a specified range**.
2. **Access time:** The time it takes to find a particular data item, or set of items, using the technique in question.
3. **Insertion and Deletion time:** The time it takes to insert a new data item or to remove a data. This value includes the time it takes to update the index structure.
4. **Space overhead:** The additional space occupied by an index structure. Provided that the amount of additional space is moderate, it is usually worthwhile to sacrifice the space to achieve improved performance.

Basic Concept

- Indexing mechanisms used to speed up access to desired data. E.g., author catalog in library
- **Search Key** - attribute to set of attributes used to look up records in a file.
- An **index file** consists of records (called index entries) of the form:

search-key	pointer

- Index files are typically much smaller than the original file
- Two basic kinds of indices:
  i. **Ordered indices**: search keys are stored in sorted order
  ii. **Hash indices**: search keys are distributed uniformly across "buckets" using a "hash function".

Search Key

Search Criteria

An attribute or set of attributes used to look up records in a file is called a search key. Note that this definition of key differs from that used in primary key, candidate key, and superkey. Multiple search keys are common in use.

Ordered Index: Types

```
1. create a clustered index for student table on
attribute ID.
2. the table gets sorted according to the clustered
index.
```

```
Clustered index

index (ID)   | Name
1            | Tawfiq
2            | Ishmam
3            | Siddique
```

A file may have several indices, on di⍰erent search keys. Ordered index can be of 2 types:

1. **Clustering Index:** Cluster index is a type of index which sorts the data rows in the table on their key values. In the Database, there is only one clustered index per table. A clustered index defines the order in which data is stored in the table which can be sorted in only one way. So, there can be an only a single clustered index for every table. In an RDBMS, usually, the primary key allows you to create a clustered index based on that specific column.This also called **Primary Index**.
2. **Nonclustering Indices:** A Non-clustered index stores the data at one location and indices at another location. The index contains pointers to the location of that data. A single table can have many non-clustered indexes as an index in the non-clustered index is stored in different places. Called **Secondary Index**.

(Definitions have been adopted from www.guru99.com)

Clustering Index

Definition

If the file containing the records is sequentially ordered, a clustering index is an index whose search key also defines the sequential order of the file.

Properties:

- ✓ Since both index and records are in the **same physical block**, searching is **very fast**
- ✓ Because the data can only be sorted in one way, you can have **only one clustered index per table**.
- ✓ Clustering indices are also called **Primary Indices**. (The term primary index may appear to denote an index on a primary key)
- ✓ The search key of a clustering index is **often the primary key**, although that is not necessarily the only option (other search keys may be used) **[theoretically!!!]**.
- ✓ The **leaf nodes** of a clustered index contain the **actual data rows**.

Non-Clustering Index

Definition

Indices whose search key specifies an order different from the sequential order of the file are called nonclustering indices, or secondary indices. In simple word, *a non-clustered index is stored in one place and the actual table data is stored in another.*

Properties:

- ✓ Index and records are in the **stored in different physical blocks**, searching is **a bit slower**
- ✓ Non-Clustering indices are also called **Secondary Indices**.
- ✓ It is possible to have **multiple non-clustered indexes** FOR **a single table.**

Primary and Secondary Index: Concepts

Primary Index on ID

Secondary Index on Name

ID	Name	CGPA
1	b	3.2
2	a	3.9
7	c	3.1
10	d	3.0
12	m	3.6
14	x	3.4
15	K	3.5
20	S	3.7

Students Table

Name
a
b
c
d
K
m
S
x

Dense and Sparse Index

Index Entry

An index entry, or index record, consists of a **search-key value** and **pointers** to one or more records with that value as their search-key value. The pointer to a record consists of the identifier of a disk block and an offset within the disk block to identify the record within the block.

Index Table

ID	Pointer
10	●
25	●
43	●
53	●

Database File

ID	Last name	Dept	Rank	Salary
10	David	EE	Faculty	95
12	Carlos	CS	Post doc	50
18	Frank	CS	Faculty	105
25	Ellen	Chem	Faculty	80
28	Jones	Physics	Faculty	82
32	Lopez	Physics	Grad St	25
37	James	Bio	Post doc	40
43	Edward	EE	Post doc	45
48	Mary	EE	Grad St	22
50	Diaz	CS	Grad St	25
53	Evans	Bio	Faculty	88

Dense Index

Dense Index: Definition

In a dense index, **an index entry** appears **for every search-key value** in the file. In a dense clustering index, the index record contains the search-key value and a pointer to the **first data record** with that search-key value.

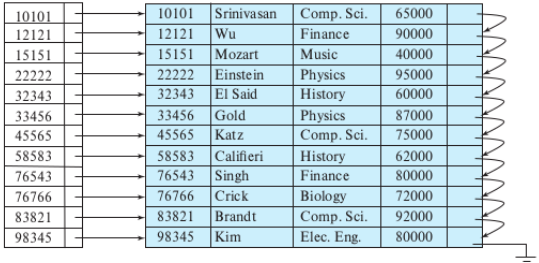


Figure: Dense Index

Sparse Index

Sparse Index: Definition

In a sparse index, an index entry appears for **only some of the search- key values**. Sparse indices can be used only if the relation is stored in **sorted order** of the search key; that is, if the index is a clustering index.

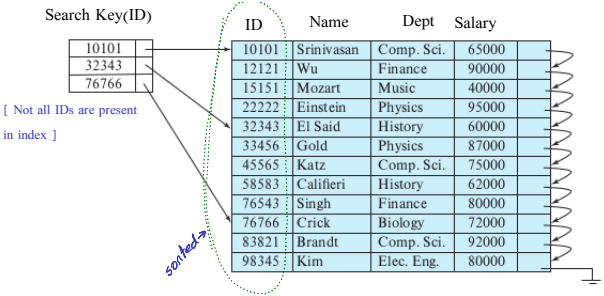


Figure: Dense Index

Dense and Sparse Index: Pros and Cons

Dense Index:

- It is generally **faster** to locate a record if we have a dense index rather than a sparse index

Sparse Index:

- Sparse indices have advantages over dense indices in that they **require less space** and they impose **less maintenance overhead** for insertions and deletions.

Access Time and Space Overhead: A trade-off

- There is a trade-off that the system designer must make between access time and space overhead.
- Trade-off depends on the specific application
- The **dominant** cost in processing a database request is the time that it takes to **bring a block from disk into main memory**.
- Once we have brought in the block, the time to scan the entire block is negligible.

Access Time and Space Overhead: A trade-off

- There is a trade-off that the system designer must make between access time and space overhead.
- Trade-off depends on the specific application
- The **dominant** cost in processing a database request is the time that it takes to **bring a block from disk into main memory**.
- Once we have brought in the block, the time to scan the entire block is negligible.

Access Time and Space Overhead: A trade-off

- There is a trade-off that the system designer must make between access time and space overhead.
- Trade-off depends on the specific application
- The **dominant** cost in processing a database request is the time that it takes to **bring a block from disk into main memory**.
- Once we have brought in the block, the time to scan the entire block is negligible.

Access Time and Space Overhead: A trade-off

- There is a trade-off that the system designer must make between access time and space overhead.
- Trade-off depends on the specific application
- The **dominant** cost in processing a database request is the time that it takes to **bring a block from disk into main memory**.
- Once we have brought in the block, the time to scan the entire block is negligible.

Multilevel Indices: Motivation

- If an index is **small** enough to be **kept entirely in main memory**, the search time to find an entry is low.
- However, for larger size index, the search for an entry in the index then requires **several disk-block reads**.
- Binary search can be used on the index file to locate an entry, but the search still has a large cost. If the index would occupy **b** blocks, binary search requires as many as $log_2 b$ blocks to be read. Even this is **quite high for a very large database**.
- To deal with this problem, we treat the index just as we would treat any other sequential file, and we construct a **sparse outer index** on the original index, which we now call the inner index.

Multilevel Indices: Motivation

- If an index is **small** enough to be **kept entirely in main memory**, the search time to find an entry is low.
- However, for larger size index, the search for an entry in the index then requires **several disk-block reads**.
- Binary search can be used on the index file to locate an entry, but the search still has a large cost. If the index would occupy **b** blocks, binary search requires as many as $log_2 b$ blocks to be read. Even this is **quite high for a very large database**.
- To deal with this problem, we treat the index just as we would treat any other sequential file, and we construct a **sparse outer index** on the original index, which we now call the inner index.

Multilevel Indices: Motivation

- If an index is **small** enough to be **kept entirely in main memory**, the search time to find an entry is low.
- However, for larger size index, the search for an entry in the index then requires **several disk-block reads**.
- Binary search can be used on the index file to locate an entry, but the search still has a large cost. If the index would occupy **b** blocks, binary search requires as many as $log_2 b$ blocks to be read. Even this is **quite high for a very large database**.
- To deal with this problem, we treat the index just as we would treat any other sequential file, and we construct a **sparse outer index** on the original index, which we now call the inner index.

Multilevel Indices: Motivation

- If an index is **small** enough to be **kept entirely in main memory**, the search time to find an entry is low.
- However, for larger size index, the search for an entry in the index then requires **several disk-block reads**.
- Binary search can be used on the index file to locate an entry, but the search still has a large cost. If the index would occupy **b** blocks, binary search requires as many as $log_2 b$ blocks to be read. Even this is **quite high for a very large database**.
- To deal with this problem, we treat the index just as we would treat any other sequential file, and we construct a **sparse outer index** on the original index, which we now call the inner index.

Multilevel Indices (Cont.): A simple Analogy

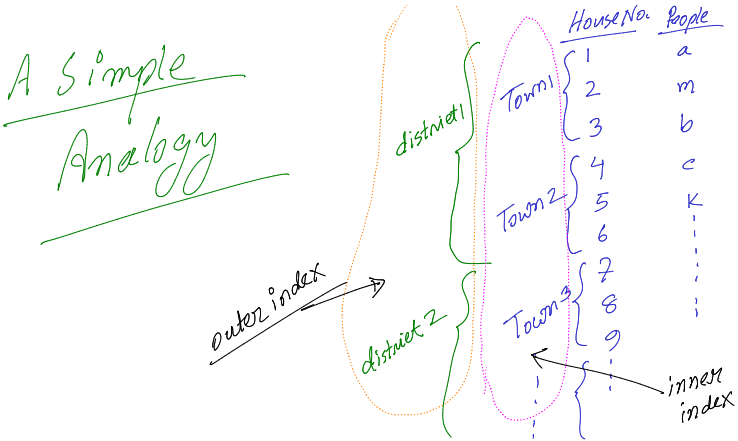


Figure: Multi-level index: Analogy

Multilevel Indices (Cont.):

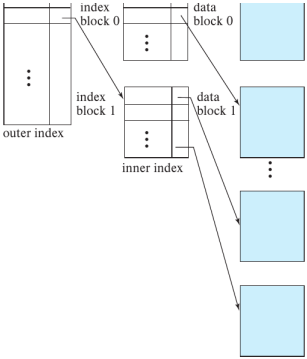


Figure: Multi-level index: Basic Idea

Multilevel Indices (Cont.):

Binary Search Vs multi-level search

In binary search tree the number of read operation to search one item is bounded by $log_2 n$ where n is the number of items in the storage. Actually binary search tree is just a special case with $m = 2$, the higher the value of m the lower the height of the tree, the faster the search. In database the term m which is eventually the number of maximum children per node is often termed as **Fan Out**.

Example: Suppose a table has 1,000,000 tuples, each block (of size 4K) contains 100 records. Hence total number of block needed is $\frac{1,000,000}{100} = 10000$

Value of m	No. of reads needed
m=2	$log_m(10000) = 13.44 = 14$
m=5	$log_m(10000) = 5.7 = 6$
m=10	$log_m(10000) = 4$
m=50	$log_m(10000) = 2.35 = 3$

B/B+ Tree

To cover this topic we need to recap few concepts:

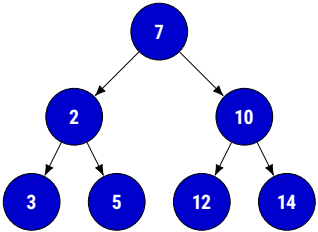
- How data is stored on disk (Blocks vs records)
- m-way search trees
- B Trees
- B+ Trees

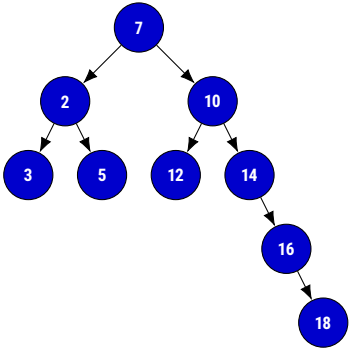
Blocks vs records

In a database, data is not read one record at a time. It is read in **blocks (or pages)**. A single block typically contains many records. So, the smallest unit of storage access is blocks. The size of each block in Postgre SQL is 8K while it also supports 4K in linux system.

The goal of indexing is to **reduce the number of block read** to fetch records upon user request.

m-way Search Tree





- In BST we have **1 key and 2 children**.
- In m-way Search Tree it may have **more than** 1 key and more than 2 children

m-way Search Tree (Cont.)

Properties:

- Node Capacity: Each node can contain at most $(m-1)$ **keys** and m **children**.
- Key Arrangement: The keys within a single node are always stored in sorted order (e.g., $K_1 < K_2 < K_3 \ldots$).

Pending: 2 figures will be placed here.

m-way Search Tree (Cont.)

Can we use m-way Search Tree data structure to store index of a database?

Ans:Yes. But it requires the following **modification**.

- It will have Record Pointer (Rp) for each corresponding Key values.
- Hence the structure becomes:
  - ✓ $m - 1$ keys
  - ✓ m child pointers
  - ✓ $m - 1$ record pointers (newly added)
- **But** in m-way Search Tree (also for BST) the tree cab grow in one direction. It is **not a balanced tree** in that case.
- And the resulting data-structure is called **Balanced Tree** or simply **B Tree**.

m-way Search Tree (Cont.)

A node now will have additional pointers called Record Pointer to map to the original database record. It looks like:

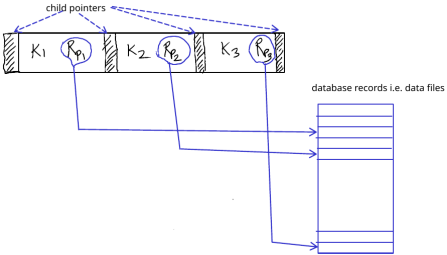


Figure: Multi-level index: Node Structure

B-Tree

Hence B-Tree has the identical node structure of a m-way Search Tree and additional Rules:

Node Structure:

- ✓ $m - 1$ keys
- ✓ m child pointers
- ✓ $m - 1$ record pointers

Set of Rules:

- ✓ $\lceil \frac{m}{2} \rceil$ children for other nodes
- ✓ Root can have minimum of 2 children and 1 key
- ✓ All leaves are at the same levels (Balanced Tree)
- ✓ Bottom-up process is followed to construct the tree

B-Tree Properties: Summary

For **m-way** B Tree: (suppose m=5)

- For no. of children
  - ✓ Max no. of children m i.e. 5
  - ✓ Min no. of children $\lceil \frac{m}{2} \rceil$ i.e. 3
- For no. of keys
  - ✓ Max no. of keys $m - 1$ i.e. 4
  - ✓ Min no. of keys $\lceil \frac{m}{2} \rceil - 1$ i.e. 2
- Root Special Case
  - ✓ Min no. of keys 1
  - ✓ Min no. of children 2

Credit: Some examples for B Tree and B+ Tree have been taken from `jennyslectures.com`

B-Tree Node Splitting

This is the fundamental step in construction of a B-Tree. **The key values (including the new one) will be sorted and then the mid value will be selected to pick it up as the immediate root.**
Node Splitting occurs when there is no space to hold another item. It is done in the following way:

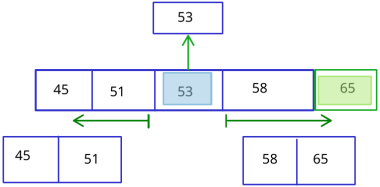
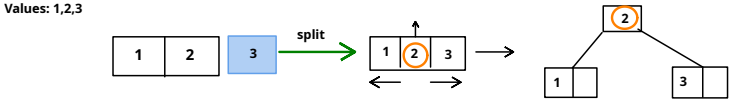


Figure: Node Splitting

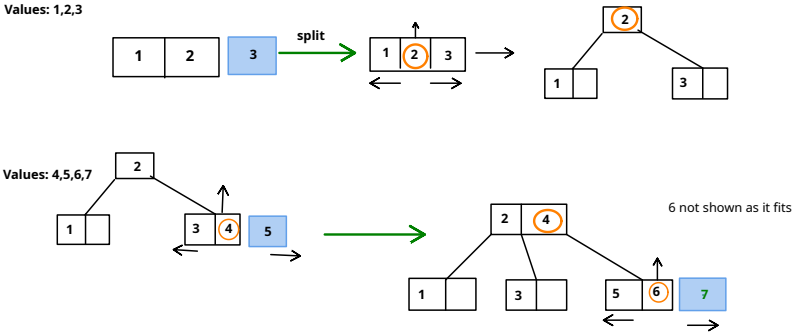
B Tree Construction: Example 1

Lets consider the B Tree with $m = 3$ (i.e. 3 max children, and 2 keys) for the data set {1,2,3,4,5,6,7,8,9,10}



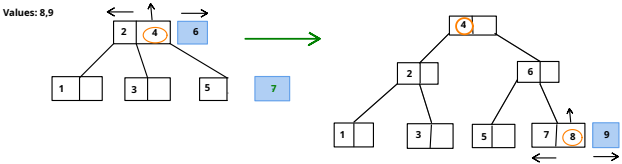
B Tree Construction: Example 1

Lets consider the B Tree with $m = 3$ (i.e. 3 max children, and 2 keys) for the data set {1,2,3,4,5,6,7,8,9,10}



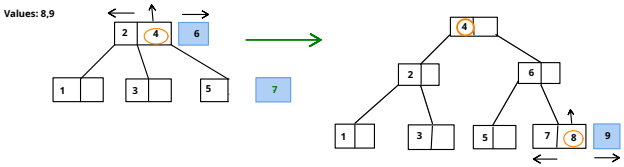
B Tree Construction: Example 1 (Cont.)

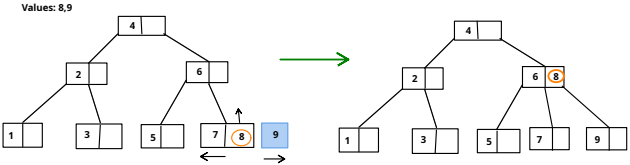
Lets consider the B Tree with $m = 3$ (i.e. 3 max children, and 2 keys) for the data set {1,2,3,4,5,6,7,8,9,10}



B Tree Construction: Example 1 (Cont.)

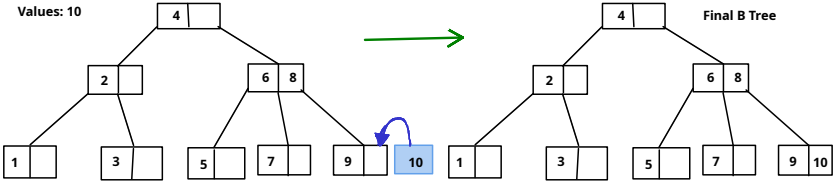
Lets consider the B Tree with $m = 3$ (i.e. 3 max children, and 2 keys) for the data set {1,2,3,4,5,6,7,8,9,10}





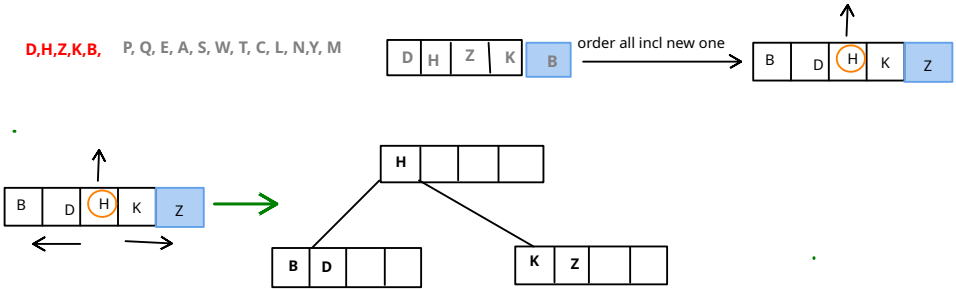
B Tree Construction: Example 1 (Cont.2)

Lets consider the B Tree with $m = 3$ (i.e. 3 max children, and 2 keys) for the data set {1,2,3,4,5,6,7,8,9,10}



B Tree Construction: Example 2

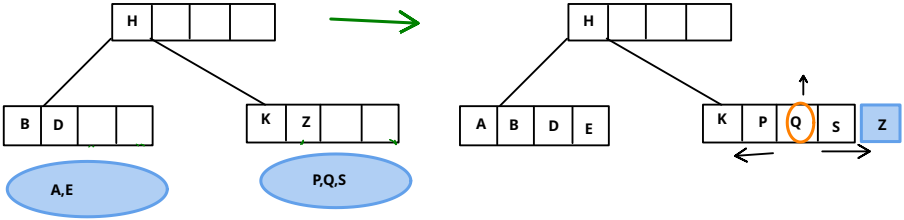
Lets consider the B Tree with $m = 5$ (i.e. 5 max children, and 4 keys) for the data set {D,H,Z,K,B,P, Q, E, A, S, W, T, C, L, N,Y, M}



B Tree Construction: Example 2 (Cont.1)

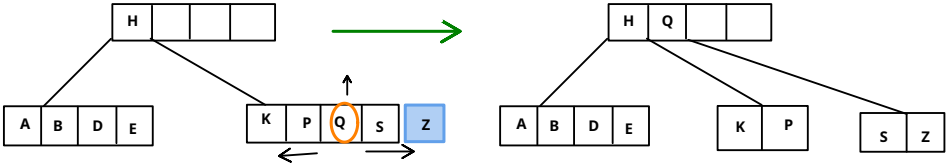
Lets consider the B Tree with $m = 5$ (i.e. 5 max children, and 4 keys) for the data set {D,H,Z,K,B,P, Q, E, A, S, W, T, C, L, N,Y, M}

D,H,Z,K,B, **P, Q, E, A, S,** W, T, C, L, N,Y, M



B Tree Construction: Example 2 (Cont.2)

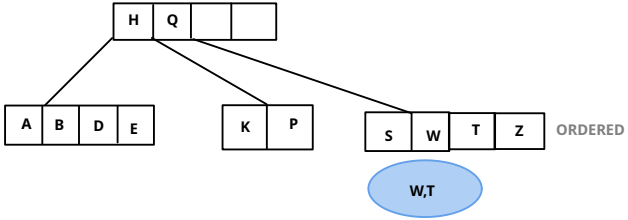
Lets consider the B Tree with $m = 5$ (i.e. 5 max children, and 4 keys) for the data set {D,H,Z,K,B,P, Q, E, A, S, W, T, C, L, N,Y, M}



B Tree Construction: Example 2 (Cont.3)

Lets consider the B Tree with $m = 5$ (i.e. 5 max children, and 4 keys) for the data set {D,H,Z,K,B,P, Q, E, A, S, W, T, C, L, N,Y, M}

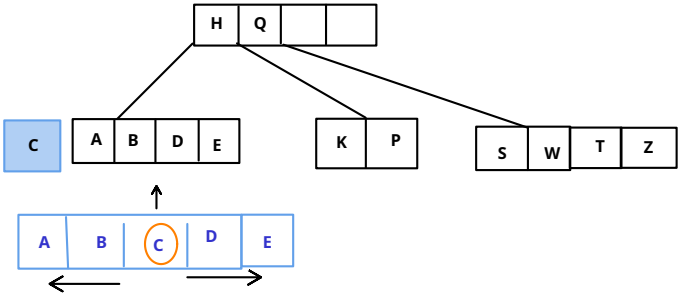
D,H,Z,K,B, P, Q, E, A, S, W, T, C, L, N,Y, M (Can be fit in one node)



B Tree Construction: Example 2 (Cont.4)

Lets consider the B Tree with $m = 5$ (i.e. 5 max children, and 4 keys) for the data set {D,H,Z,K,B,P, Q, E, A, S, W, T, C, L, N,Y, M}

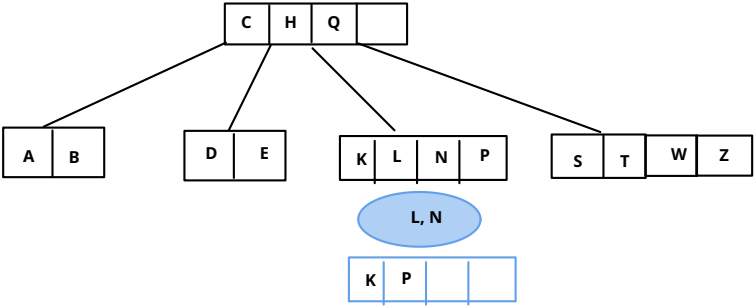
D,H,Z,K,B, P, Q, E, A, S, W, T, C , L, N,Y, M



B Tree Construction: Example 2 (Cont.5)

Lets consider the B Tree with $m = 5$ (i.e. 5 max children, and 4 keys) for the data set {D,H,Z,K,B,P, Q, E, A, S, W, T, C, L, N,Y, M}

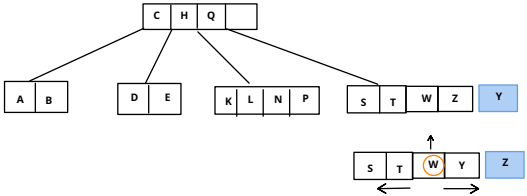
D,H,Z,K,B, P, Q, E, A, S, W, T, C , L, N, Y, M

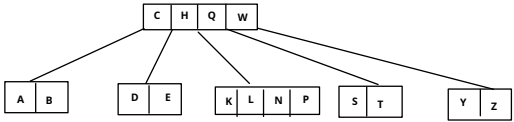


B Tree Construction: Example 2 (Cont.6)

Lets consider the B Tree with $m = 5$ (i.e. 5 max children, and 4 keys) for the data set {D,H,Z,K,B,P, Q, E, A, S, W, T, C, L, N,Y, M}

D,H,Z,K,B, P, Q, E, A, S, W, T, C , L, N, Y ,M

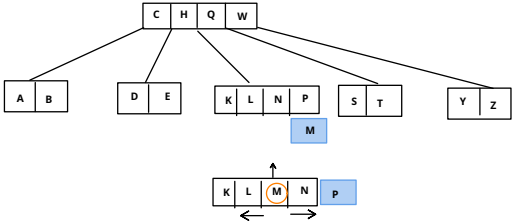




B Tree Construction: Example 2 (Cont.7)

Lets consider the B Tree with $m = 5$ (i.e. 5 max children, and 4 keys) for the data set {D,H,Z,K,B,P, Q, E, A, S, W, T, C, L, N,Y, M}

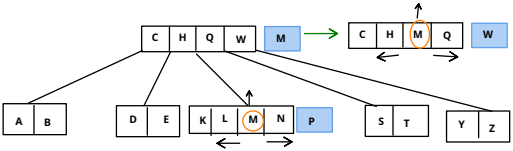
D,H,Z,K,B, P, Q, E, A, S, W, T, C , L, N, Y ,M



B Tree Construction: Example 2 (Cont.8)

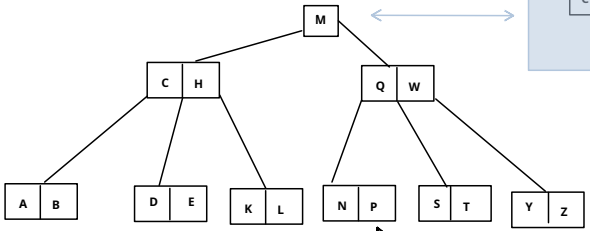
Lets consider the B Tree with $m = 5$ (i.e. 5 max children, and 4 keys) for the data set {D,H,Z,K,B,P, Q, E, A, S, W, T, C, L, N,Y, M}

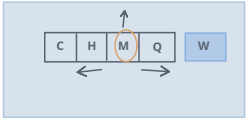
D,H,Z,K,B, P, Q, E, A, S, W, T, C , L, N, Y ,M



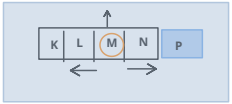
B Tree Construction: Example 2 (Cont.9) and END

D,H,Z,K,B, P, Q, E, A, S, W, T, C , L, N, Y ,M





Note: N,P are placed left of Q, as per rule, start from M



Cases on Deletion of B Tree

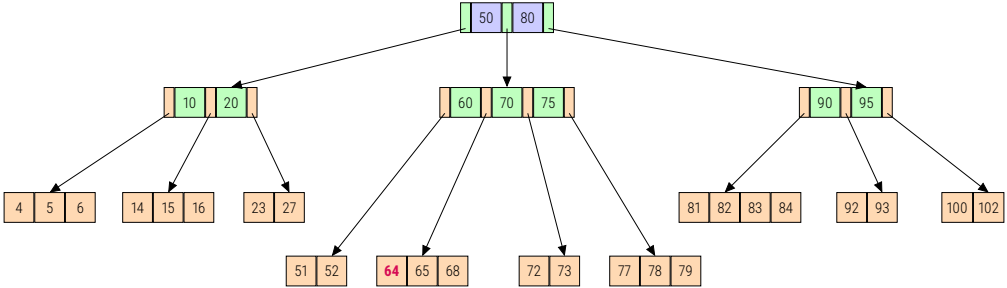
- B Tree **Node Deletion**
  - (1) Leaf Node
    - (a) Nodes with more than Min Keys
    - (b) Nodes with exactly Min Keys
      - (i) Borrow from Left Sibling
      - (ii) Borrow from Right Sibling
      - (iii) Not possible to borrow from either
  - (2)Internal Node
    - (a) Nodes with more than Min Keys
    - (b) Nodes with exactly Min Keys

B-Tree Deletion: Case 1.(a)

Here **m=5** i.e. Max Children 5 and Max Keys 4

Deletion List: **64**

Action: Just delete it since there are enough keys even after deleting this, so it incurs no extra process of adjustment.

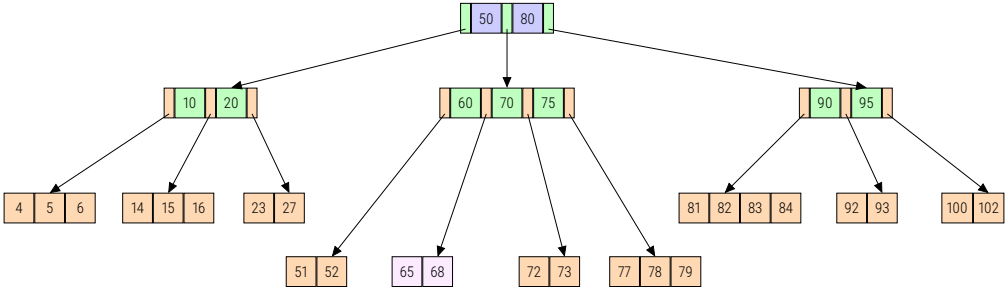


B-Tree Deletion: Case 1.(a)

Here **m=5** i.e. Max Children 5 and Max Keys 4

Deletion List: **64**

Action: Just delete it since there are enough keys even after deleting this, so it incurs no extra process of adjustment.

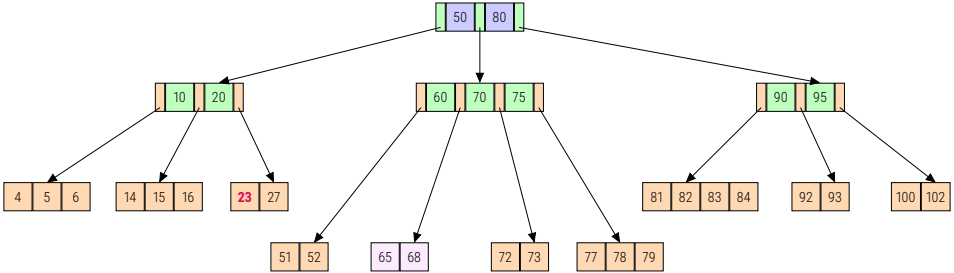


Same is the case for 16/77/84 and so on.

B-Tree Deletion: Case 1.(b)(i)

Deletion List: 64,**23** (Here **m=5** i.e. Max Children 5 and Max Keys 4)

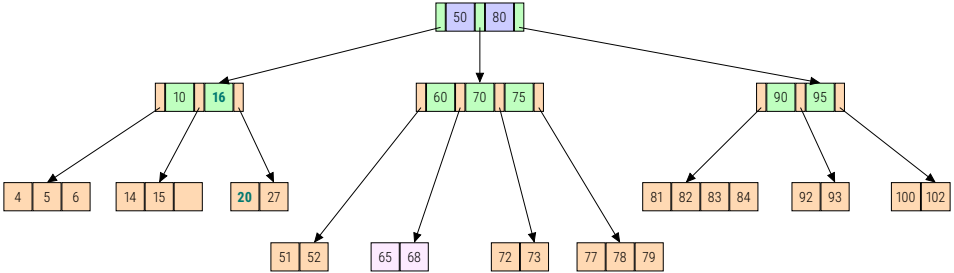
Algorithm: From Left Sibling **max value** will go up to its immediate root (iroot) and the root will be replaced by the key value (i.e. delete item). Here **16 will go up** and send the iroot i.e. 20 (simply **20 will go down**) in place of the delete item i.e. 23 (simply 23 will be removed).



B-Tree Deletion: Case 1.(b)(i)

Deletion List: 64,**23** (Here **m=5** i.e. Max Children 5 and Max Keys 4)

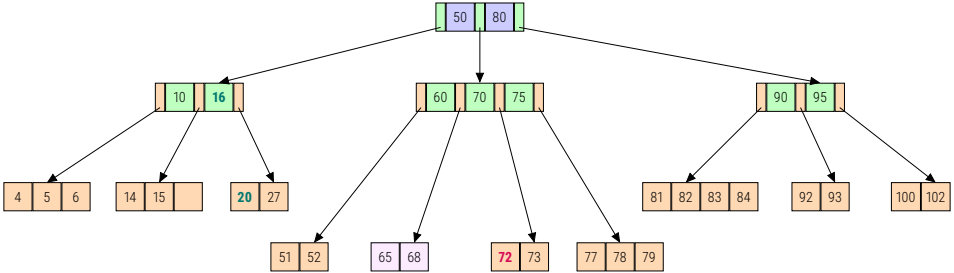
Algorithm: From Left Sibling **max value** will go up to its immediate root (iroot) and the root will be replaced by the key value (i.e. delete item). Here **16 will go up** and send the iroot i.e. 20 (simply **20 will go down**) in place of the delete item i.e. 23 (simply 23 will be removed).



B-Tree Deletion: Case 1.(b)(ii)

Deletion List: 64,23,**72** (Here **m=5** i.e. Max Children 5 and Max Keys 4)

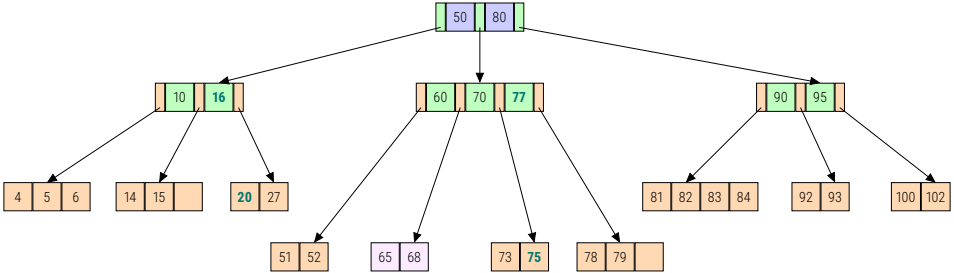
Algorithm: From Right Sibling **min value** will go up to its immediate root (iroot) and the root will be replaced by the key value (i.e. delete item). Here **77 will go up** and send the iroot i.e. 75 (simply **75 will go down**) in place of the delete item i.e. 72 (simply 72 will be removed).



B-Tree Deletion: Case 1.(b)(ii)

Deletion List: 64,23,**72** (Here **m=5** i.e. Max Children 5 and Max Keys 4)

Algorithm: From Right Sibling **min value** will go up to its immediate root (iroot) and the root will be replaced by the key value (i.e. delete item). Here **77 will go up** and send the iroot i.e. 75 (simply **75 will go down**) in place of the delete item i.e. 72 (simply 72 will be removed).

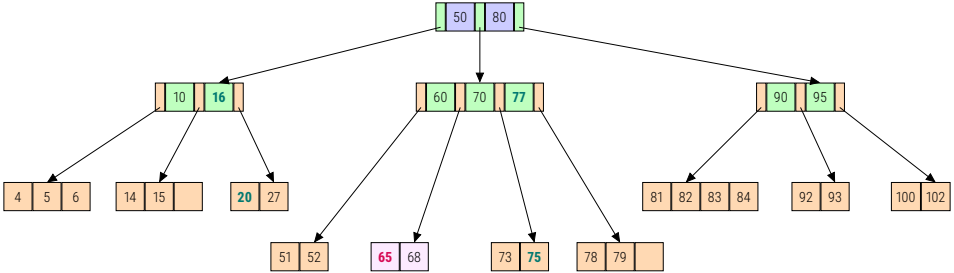


B-Tree Deletion: Case 1.(b)(iii)

Deletion List: 64,23,72,**65** (Here **m=5** i.e. Max Children 5 and Max Keys 4)

Algorithm: Either Left or Right merge with immediate parent node merging. (Left one is shown here)

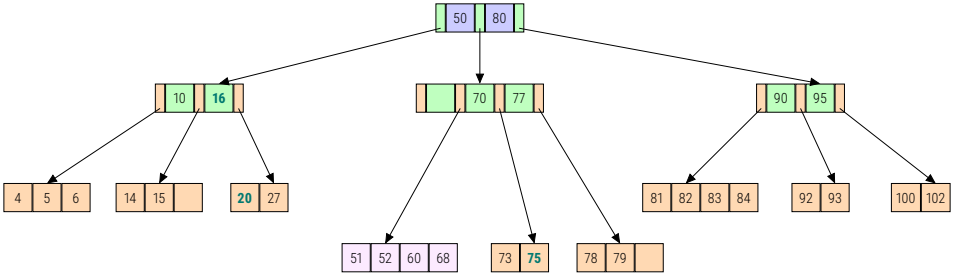
Both node (51,52) and node (-,68) will be merged with 60 as it is the immediate parent node.



B-Tree Deletion: Case 1.(b)(iii)

Deletion List: 64,23,72,**65** (Here **m=5** i.e. Max Children 5 and Max Keys 4)

Algorithm: Either Left or Right merge with immediate parent node merging. (Left one is shown here)
Both node (51,52) and node (-,68) will be merged with 60 as it is the immediate parent node.



B-Tree to B+ Tree Conversion

- In a **B-Tree**: Keys and actual records (or record pointers) can be stored in both internal and leaf nodes.
  While in a **B+ Tree**: All actual records are stored only in leaf nodes. Internal nodes store only keys (as separators).
- In a **B-tree**: **Each key** appears **only once**. But in B+ Tree, Keys in internal nodes are copies of leaf keys (stored more than once).
- Furthermore, In a B+ tree: **All leaf nodes** are connected using a **linked list** (usually right pointer). This makes range queries very efficient.
- Another property: nodes at the right are greater **or equal** to its parent node.

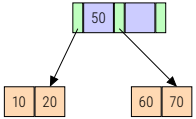


Figure: B Tree

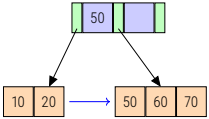
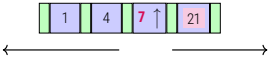
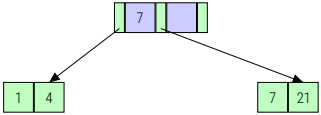


Figure: B+ Tree

B+-Tree Construction Selective Steps (a)

Dataset: 1,4,7,10,17,21,31,25,19,20,28,42 $m = 4$ i.e. Max Child 4, **Max Key 3**

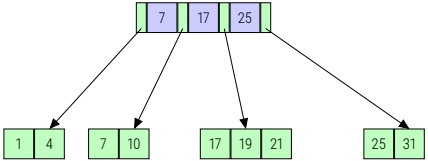




Note: This frame demonstrates custom colors (blue/green for internal nodes, orange for leaves) and disabled asterisks using `\btreeasteriskoff`.

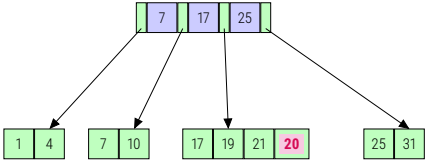
B+-Tree Construction Selective Steps (b)

Dataset: 1,4,7,10,17,21,31,25,19,**20**,28,42 $m = 4$ i.e. Max Child 4, **Max Key 3**



B+-Tree Construction Selective Steps (b)

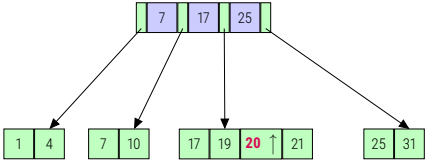
Dataset: 1,4,7,10,17,21,31,25,19,**20**,28,42 $m = 4$ i.e. Max Child 4, **Max Key 3**



- Splitting is required, but before that they must be sorted.
- After sorting the middle key value will be lifted up as part of splitting

B+-Tree Construction Selective Steps (b)

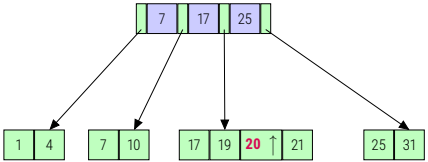
Dataset: 1,4,7,10,17,21,31,25,19,**20**,28,42 $m = 4$ i.e. Max Child 4, **Max Key 3**

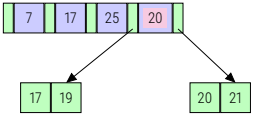


- Splitting is required, but before that they must be sorted.
- After sorting the middle key value will be lifted up as part of splitting

B+-Tree Construction Selective Steps (b)

Dataset: 1,4,7,10,17,21,31,25,19,**20**,28,42 $m = 4$ i.e. Max Child 4, **Max Key 3**

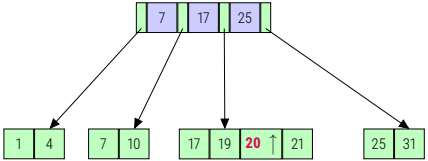


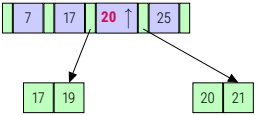


- Splitting is required, but before that they must be sorted.
- After sorting the middle key value will be lifted up as part of splitting

B+-Tree Construction Selective Steps (b)

Dataset: 1,4,7,10,17,21,31,25,19,**20**,28,42 $m = 4$ i.e. Max Child 4, **Max Key 3**

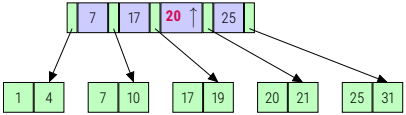




- Splitting is required, but before that they must be sorted.
- After sorting the middle key value will be lifted up as part of splitting

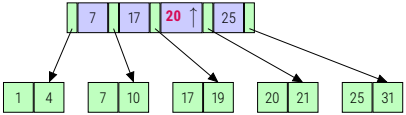
B+-Tree Construction Selective Steps (c)

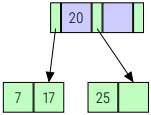
Dataset: 1,4,7,10,17,21,31,25,19,**20**,28,42 $m = 4$ i.e. Max Child 4, **Max Key 3**



B+-Tree Construction Selective Steps (c)

Dataset: 1,4,7,10,17,21,31,25,19,**20**,28,42 $m = 4$ i.e. Max Child 4, **Max Key 3**

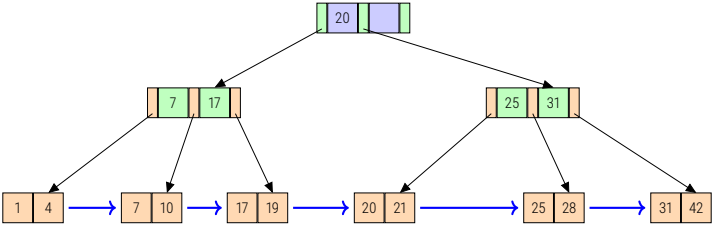




- Splitting is required, but before that they must be sorted.
- After sorting the middle key value will be lifted up as part of splitting

Final B+-Tree

Dataset: 1,4,7,10,17,21,31,25,19,20,28,42

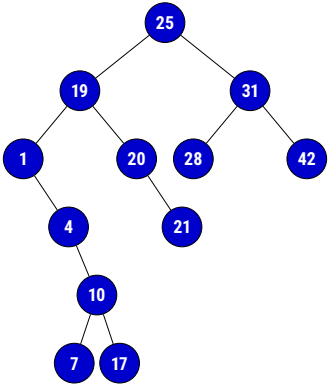


BST, B Tree and B+ Tree in Database Query

- BST is not suitable for handling large values which is common in database
- B Tree is very fast for single point SQL
- B+ Tree is very fast for Range Query
- We will visualize it, hence we will draw the BST and B Tree for the same dataset and m value

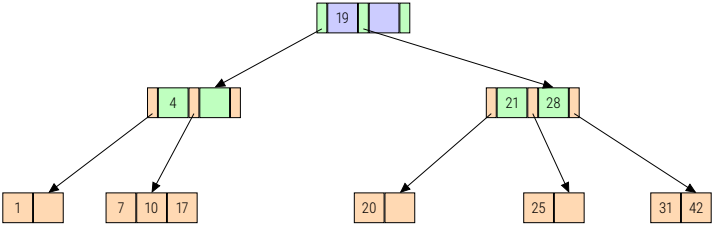
BST for the Same dataset

Dataset: 25,19,20,1,4,10,17,21,31,28,42,7 (the order determines the skew of the tree)



B Tree for the Same dataset with $m = 4$

Dataset: 25,19,20,1,4,10,17,21,31,28,42,7



Indexing and SQL

Dataset: 25,19,20,1,4,10,17,21,31,28,42,7 (the order determines the skew of the tree)

Now consider the following SQL

```
SELECT ADDRESS
FROM EMP
WHERE ID=21;
```

Now lets count the no. of comparison needed to execute the query in 3 cases:

1. In BST it is 4
2. In B Tree it is: 2
3. In B+ Tree it is: 3

Indexing and SQL (Cont.)

Dataset: 25,19,20,1,4,10,17,21,31,28,42,7 (the order determines the skew of the tree)

Now consider the following SQL

```
SELECT ADDRESS
FROM EMP
WHERE ID BETWEEN 7 AND 21;
```

Now lets count the no. of comparison needed to execute the query in 3 cases:
The range covers the values: **7,10,17,19,20.21**

1. In BST it is 6+5+6+2+3+4=26
2. In B Tree it is: 3+0+0+2+3+2=10
3. In B+ Tree it is: 2 (**only**)

Bitmap Indices

Defintion

A **bitmap** is simply an array of bits. In its simplest form, a bitmap index on the attribute A of relation r consists of one bitmap for each value that A can take. Each bitmap has **as many bits as the number of records** in the relation.

Bitmap Indices

Defintion

A **bitmap** is simply an array of bits. In its simplest form, a bitmap index on the attribute A of relation r consists of one bitmap for each value that A can take. Each bitmap has **as many bits as the number of records** in the relation.

Where to use?

It has an efficient implementation when total number of distinct values of the selected attribute is limited. Example: Gender, Blood Group, or any customized class. Class must be non-overlapping.

Bitmap Indices: General Idea

record number	*ID*	*gender*	*income_level*
0	76766	m	L1
1	22222	f	L2
2	12121	f	L1
3	15151	m	L4
4	58583	f	L3

Bitmaps for *gender*

m	10010
f	01101

Bitmaps for *income_level*

L1	10100
L2	01000
L3	00001
L4	00010
L5	00000

Bitmap Indices: General Idea

record number	*ID*	*gender*	*income_level*
0	76766	m	L1
1	22222	f	L2
2	12121	f	L1
3	15151	m	L4
4	58583	f	L3

Bitmaps for *gender*

m	10010
f	01101

Bitmaps for *income_level*

L1	10100
L2	01000
L3	00001
L4	00010
L5	00000

Now consider the following SQL:

select ID
from citizens
where gender='m' and income=L1;

Now we can readily get the result (i.e. index of record) by and-ing the vectors. You will special benefit when you need count(*) function. No need to look for the records even, only bit vectors can compute the result.

Test of cross symbol

$AB\not\subset DEF+ = \{A, B, C, D, E\}$ (since $A \rightarrow C$)